МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 4

Тема: **Структурные паттерны: Компоновщик, Декоратор**

Дисциплина: **«Паттерны проектирования»**

Выполнила:

Студентка группы ПИЖ-б-о-23-2
направление подготовки 09.03.04
«Программная инженерия»

Силина Оксана Романовна

Проверил:

Ассистент ДЦРСИЭ

Белик А. И.

Ставрополь 2026

Цель работы: изучить принцип работы структурных паттернов проектирования «Компоновщик» (Composite) и «Декоратор» (Decorator) и реализовать прикладную задачу расчёта траектории движения тела с использованием данных паттернов.

Задача: разработать приложение для расчёта параметров движения тела, брошенного под углом к горизонту, с использованием двух структурных паттернов проектирования.

Выбранные паттерны:

- Компоновщик — для объединения нескольких участков траектории в единую составную траекторию;
- Декоратор — для динамического добавления к модели движения дополнительных эффектов (сопротивление воздуха, логирование расчётов).

Индивидуальный вариант: 3. Разработка приложения «Траектория движения»

Репозиторий: https://github.com/tsukuri1/patterns

1. Описание паттернов

1.1. Компоновщик (Composite)

Компоновщик — это структурный паттерн проектирования, который позволяет объединять объекты в древовидные структуры и работать с ними так же, как с отдельными объектами. Паттерн вводит общий интерфейс для «листьев» (простых объектов) и «контейнеров» (составных объектов).

Благодаря компоновщику пользователь может собрать траекторию из нескольких участков (например, тело движется по этапам с разными начальными условиями) и получить суммарные параметры: общее время, максимальную высоту и общую дальность. При этом клиентский код не различает одиночный участок и составную траекторию.

Применение в задаче:

Интерфейс TrajectoryComponent задаёт методы points(), total_time(), max_height(), flight_range(). Класс SimpleSegment — «лист», описывающий

один участок траектории (например, полёт от точки A до точки B). Класс CompositeTrajectory — «контейнер», объединяющий несколько участков и вычисляющий суммарные параметры. Клиентский код работает с обоими через общий интерфейс.

1.2. Декоратор (Decorator)

Декоратор — это структурный паттерн проектирования, который позволяет динамически добавлять объекту новое поведение, не изменяя его класс. Декоратор оборачивает объект и делегирует ему вызовы, добавляя свою логику до или после.

В приложении реализованы два декоратора. DragDecorator добавляет к выбранной модели учёт сопротивления воздуха, а LoggingDecorator записывает все вызовы методов модели в лог. Декораторы можно комбинировать: например, обернуть идеальную модель сначала в DragDecorator, а затем в LoggingDecorator. Это позволяет пользователю включать нужные эффекты независимо от выбранной базовой модели.

Применение в задаче:

Интерфейс MotionModel описывает модель движения (flight_time(), max_height(), flight_range(), trajectory_points()). Базовые модели — IdealModel и BasicDragModel. Декораторы DragDecorator и LoggingDecorator оборачивают любую модель и добавляют к ней соответственно учёт сопротивления воздуха и запись лога расчётов. Декораторы можно комбинировать, навешивая их друг на друга.

2. Структура приложения

Приложение состоит из следующих компонентов:

Компонент	Назначение
TrajectoryComponent	Общий интерфейс для листа и контейнера (Компоновщик)
SimpleSegment	Лист — один участок траектории

CompositeTrajectory	Контейнер — объединение нескольких участков
MotionModel	Базовый интерфейс модели движения (Декоратор)
IdealModel	Базовая модель без сопротивления
BasicDragModel	Базовая модель с упрощённым сопротивлением
MotionDecorator	Абстрактный декоратор
DragDecorator	Декоратор, добавляющий сопротивление воздуха
LoggingDecorator	Декоратор, добавляющий логирование расчётов
TrajectoryApp	Графический интерфейс приложения

3. Диаграмма классов

Для отображения структуры разработанного приложения была создана UML-диаграмма классов.

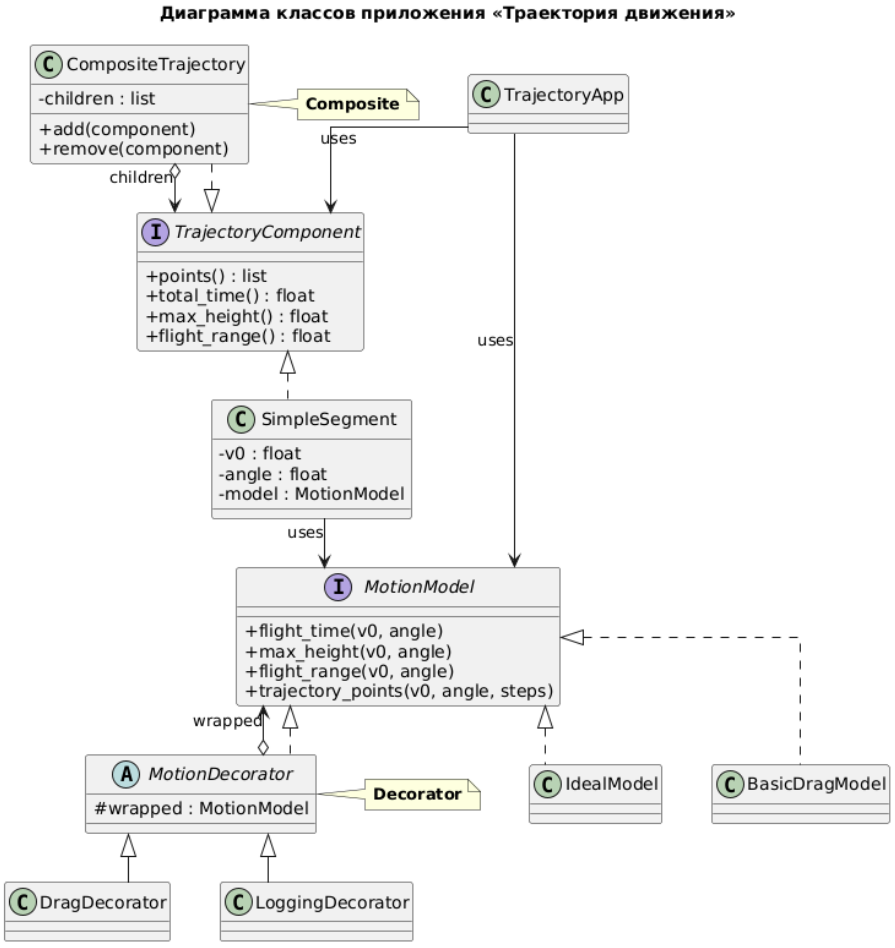


Рисунок 1 — Диаграмма классов приложения «Траектория движения»

Диаграмма содержит:

- Интерфейс TrajectoryComponent, лист SimpleSegment и контейнер CompositeTrajectory (паттерн «Компоновщик»);
- Интерфейс MotionModel, базовые модели и декораторы DragDecorator, LoggingDecorator (паттерн «Декоратор»);
- Класс TrajectoryApp — интерфейс приложения.

4. Реализация паттернов

4.1. Паттерн «Компоновщик»

TrajectoryComponent задаёт общий интерфейс. SimpleSegment — лист, описывающий один участок с заданной скоростью, углом и моделью движения. CompositeTrajectory — контейнер, хранящий список дочерних компонентов (могут быть как листья, так и другие контейнеры) и вычисляющий суммарные параметры через обход всех детей.

```
class TrajectoryComponent:
    """Общий интерфейс для листа и контейнера."""
    def points(self):
        raise NotImplementedError

    def total_time(self):
        raise NotImplementedError

    def max_height(self):
        raise NotImplementedError

    def flight_range(self):
        raise NotImplementedError


class SimpleSegment(TrajectoryComponent):
    """Лист — один участок траектории."""
    def __init__(self, v0, angle, model):
        self.v0 = v0
```

```
        self.angle = angle
        self.model = model

    def points(self):
        return self.model.trajectory_points(self.v0, self.angle, 50)

    def total_time(self):
        return self.model.flight_time(self.v0, self.angle)

    def max_height(self):
        return self.model.max_height(self.v0, self.angle)

    def flight_range(self):
        return self.model.flight_range(self.v0, self.angle)


class CompositeTrajectory(TrajectoryComponent):
    """Контейнер — объединение нескольких участков."""
    def __init__(self):
        self.children = []

    def add(self, component):
        self.children.append(component)

    def remove(self, component):
        self.children.remove(component)

    def points(self):
        result = []
        offset = 0.0
        for child in self.children:
            for (x, y) in child.points():
                result.append((round(x + offset, 3), y))
            offset += child.flight_range()

        return result

    def total_time(self):
        return sum(child.total_time() for child in self.children)

    def max_height(self):
        return max((child.max_height() for child in self.children),
default=0.0)

    def flight_range(self):
```

```
        return sum(child.flight_range() for child in self.children)
```

4.2 Паттерн «Декоратор»

MotionModel — базовый интерфейс. IdealModel — простая модель без сопротивления. MotionDecorator — абстрактный декоратор, хранящий ссылку на обёрнутую модель и делегирующий ей вызовы. DragDecorator добавляет учёт сопротивления, LoggingDecorator записывает вызовы в лог. Декораторы можно комбинировать, навешивая один на другой.

```
import math


class MotionModel:
    def flight_time(self, v0, angle):
        raise NotImplementedError

    def max_height(self, v0, angle):
        raise NotImplementedError

    def flight_range(self, v0, angle):
        raise NotImplementedError

    def trajectory_points(self, v0, angle, steps):
        raise NotImplementedError


class IdealModel(MotionModel):
    def __init__(self, g=9.81):
        self.g = g

    def flight_time(self, v0, angle):
        a = math.radians(angle)
        return 2 * v0 * math.sin(a) / self.g

    def max_height(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g)

    def flight_range(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * math.sin(2 * a) / self.g
```

```
    def trajectory_points(self, v0, angle, steps):
        T = self.flight_time(v0, angle)
        a = math.radians(angle)
        pts = []
        for i in range(steps + 1):
            t = T * i / steps
            x = v0 * math.cos(a) * t
            y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
            pts.append((round(x, 3), round(y, 3)))
        return pts


class BasicDragModel(MotionModel):
    def __init__(self, g=9.81, k=0.05):
        self.g = g
        self.k = k

    def flight_time(self, v0, angle):
        a = math.radians(angle)
        return 2 * v0 * math.sin(a) / (self.g * (1 + self.k))

    def max_height(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g * (1 + self.k))

    def flight_range(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * math.sin(2 * a) / (self.g * (1 + self.k))

    def trajectory_points(self, v0, angle, steps):
        T = self.flight_time(v0, angle)
        a = math.radians(angle)
        pts = []
        for i in range(steps + 1):
            t = T * i / steps
            x = v0 * math.cos(a) * t / (1 + self.k * t)
            y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
            pts.append((round(x, 3), round(max(y, 0), 3)))
        return pts


class MotionDecorator(MotionModel):
    """Абстрактный декоратор."""
    def __init__(self, wrapped: MotionModel):
```

```
        self.wrapped = wrapped

    def flight_time(self, v0, angle):
        return self.wrapped.flight_time(v0, angle)

    def max_height(self, v0, angle):
        return self.wrapped.max_height(v0, angle)

    def flight_range(self, v0, angle):
        return self.wrapped.flight_range(v0, angle)

    def trajectory_points(self, v0, angle, steps):
        return self.wrapped.trajectory_points(v0, angle, steps)


class DragDecorator(MotionDecorator):
    """Декоратор: добавляет учёт сопротивления воздуха."""
    def __init__(self, wrapped: MotionModel, k=0.1):
        super().__init__(wrapped)
        self.k = k

    def flight_time(self, v0, angle):
        return self.wrapped.flight_time(v0, angle) / (1 + self.k)

    def max_height(self, v0, angle):
        return self.wrapped.max_height(v0, angle) / (1 + self.k)

    def flight_range(self, v0, angle):
        return self.wrapped.flight_range(v0, angle) / (1 + self.k)


class LoggingDecorator(MotionDecorator):
    """Декоратор: добавляет логирование расчётов."""
    def __init__(self, wrapped: MotionModel):
        super().__init__(wrapped)
        self.log = []

    def flight_time(self, v0, angle):
        result = self.wrapped.flight_time(v0, angle)
        self.log.append(f"flight_time({v0}, {angle}) = {result:.3f}")
        return result

    def max_height(self, v0, angle):
        result = self.wrapped.max_height(v0, angle)
        self.log.append(f"max_height({v0}, {angle}) = {result:.3f}")
```

```
        return result

    def flight_range(self, v0, angle):
        result = self.wrapped.flight_range(v0, angle)
        self.log.append(f"flight_range({v0}, {angle}) = {result:.3f}")
        return result

    def trajectory_points(self, v0, angle, steps):
        pts = self.wrapped.trajectory_points(v0, angle, steps)
        self.log.append(f"trajectory_points({v0}, {angle}, {steps}) =
{len(pts)} точек")
        return pts

    def get_log(self):
        return self.log
```

4.3. Графический интерфейс

Интерфейс построен так, чтобы обе возможности паттернов были видны пользователю. Выбор модели и флажки «Добавить сопротивление» / «Включить логирование» демонстрируют работу декораторов: к выбранной модели динамически навешиваются обёртки. Кнопка «Добавить участок в компоновщик» позволяет собрать составную траекторию из нескольких участков, а результаты расчёта показывают суммарные параметры. Если включён LoggingDecorator, в окне выводится лог вызовов модели.

```
import tkinter as tk
from tkinter import messagebox
import matplotlib.pyplot as plt


class TrajectoryApp:
    def __init__(self, root):
        self.root = root
        root.title("Траектория движения")

        self.model = IdealModel()
        self.logging = None
        self.component = None
        self.segments = []
```

```
        tk.Label(root, text="Начальная скорость (м/с):").grid(row=0,
column=0, sticky="w")
        self.v0_entry = tk.Entry(root)
        self.v0_entry.insert(0, "20")
        self.v0_entry.grid(row=0, column=1)

        tk.Label(root, text="Угол броска (°):").grid(row=1, column=0,
sticky="w")
        self.angle_entry = tk.Entry(root)
        self.angle_entry.insert(0, "45")
        self.angle_entry.grid(row=1, column=1)

        tk.Label(root, text="Модель:").grid(row=2, column=0,
sticky="w")
        self.model_var = tk.StringVar(value="ideal")
        tk.Radiobutton(root, text="Идеальная", variable=self.model_var,
                value="ideal",
command=self.update_model).grid(row=2, column=1, sticky="w")
        tk.Radiobutton(root, text="С сопротивлением",
variable=self.model_var,
                value="drag",
command=self.update_model).grid(row=3, column=1, sticky="w")

        self.drag_var = tk.BooleanVar(value=False)
        tk.Checkbutton(root, text="Добавить сопротивление
(декоратор)",
                variable=self.drag_var).grid(row=4, column=0,
columnspan=2, sticky="w")

        self.log_var = tk.BooleanVar(value=False)
        tk.Checkbutton(root, text="Включить логирование
(декоратор)",
                variable=self.log_var).grid(row=5, column=0,
columnspan=2, sticky="w")

        tk.Button(root, text="Добавить участок в компоновщик",
            command=self.add_segment).grid(row=6, column=0,
columnspan=2, pady=3)
        tk.Button(root, text="Рассчитать",
            command=self.calculate).grid(row=7, column=0,
columnspan=2, pady=3)

        self.result = tk.Text(root, width=55, height=14)
        self.result.grid(row=8, column=0, columnspan=2)
```

```
        def update_model(self):
            base = IdealModel() if self.model_var.get() == "ideal" else
BasicDragModel()
            self.model = base

        def _build_model(self):
            """Навешиваем декораторы на базовую модель."""
            model = self.model
            if self.drag_var.get():
                model = DragDecorator(model)
            if self.log_var.get():
                self.logging = LoggingDecorator(model)
                model = self.logging
            else:
                self.logging = None
            return model

        def _read_inputs(self):
            v0 = float(self.v0_entry.get())
            angle = float(self.angle_entry.get())
            if v0 <= 0:
                raise ValueError("Скорость должна быть > 0")
            if not (0 < angle < 90):
                raise ValueError("Угол должен быть в диапазоне (0; 90)")
            return v0, angle

        def add_segment(self):
            """Добавляем очередной участок в компоновщик."""
            try:
                v0, angle = self._read_inputs()
                model = self._build_model()
                segment = SimpleSegment(v0, angle, model)
                self.segments.append(segment)
                messagebox.showinfo("Компоновщик",
                                    f"Участок добавлен. Всего участков:
{len(self.segments)}")
            except ValueError as e:
                messagebox.showerror("Ошибка ввода", str(e))

        def calculate(self):
            try:
                if not self.segments:
                    v0, angle = self._read_inputs()
                    model = self._build_model()
                    segment = SimpleSegment(v0, angle, model)
```

```
                self.segments.append(segment)

            composite = CompositeTrajectory()
            for seg in self.segments:
                composite.add(seg)

            total_time = composite.total_time()
            height = composite.max_height()
            distance = composite.flight_range()
            pts = composite.points()

            self.result.delete("1.0", tk.END)
            self.result.insert(tk.END,
                f"Участков: {len(self.segments)}\n"
                f"Суммарное время: {total_time:.3f} с\n"
                f"Макс. высота: {height:.3f} м\n"
                f"Общая дальность: {distance:.3f} м\n")

            if self.logging:
                self.result.insert(tk.END, "\nЛог декоратора:\n")
                for line in self.logging.get_log():
                    self.result.insert(tk.END, "  " + line + "\n")

            xs = [p[0] for p in pts]
            ys = [p[1] for p in pts]
            plt.plot(xs, ys, marker='o', markersize=3)
            plt.title("Составная траектория")
            plt.xlabel("X, м")
            plt.ylabel("Y, м")
            plt.grid(True)
            plt.axis('equal')
            plt.show()
        except ValueError as e:
            messagebox.showerror("Ошибка ввода", str(e))
        except Exception as e:
            messagebox.showerror("Ошибка", str(e))


if __name__ == "__main__":
    root = tk.Tk()
    app = TrajectoryApp(root)
    root.mainloop()
```

5. Результаты работы программы

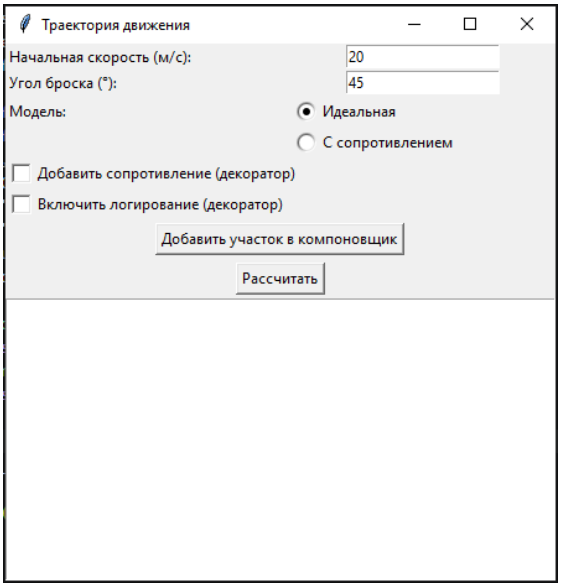


Рисунок 2 — Главное окно приложения

Пользователь вводит параметры, выбирает модель, отмечает декораторы и добавляет участки в компоновщик.

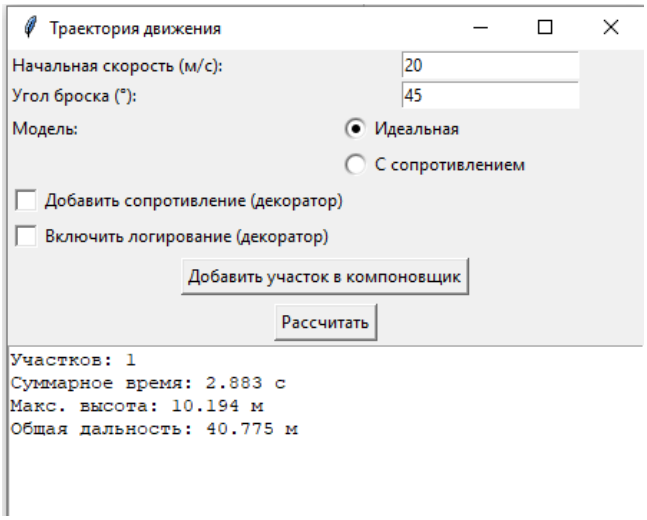


Рисунок 3 — Расчёт одного участка

При v0 = 20 м/с, угол = 45°, модель IdealModel:

- Время полёта: 2.883 с
- Макс. высота: 10.194 м
- Дальность: 40.774 м

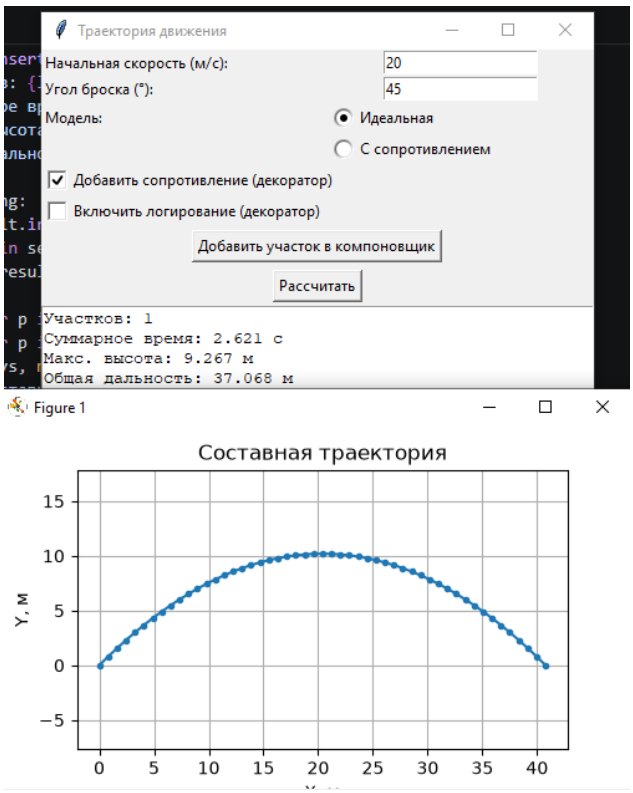


Рисунок 4 — Расчёт с декоратором DragDecorator

Дальность и высота уменьшаются пропорционально коэффициенту сопротивления, при этом код IdealModel не изменяется.

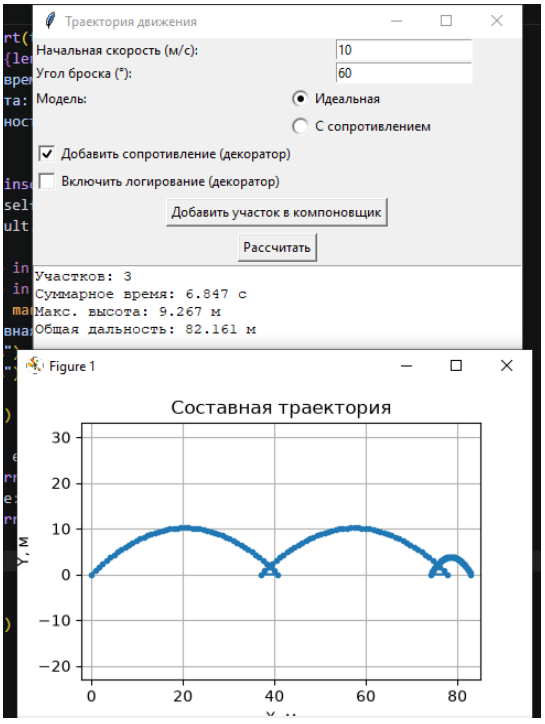


Рисунок 5 — Расчёт составной траектории (Компоновщик)

Несколько участков объединены в CompositeTrajectory; суммарное время и дальность складываются, максимальная высота берётся как максимум по участкам.

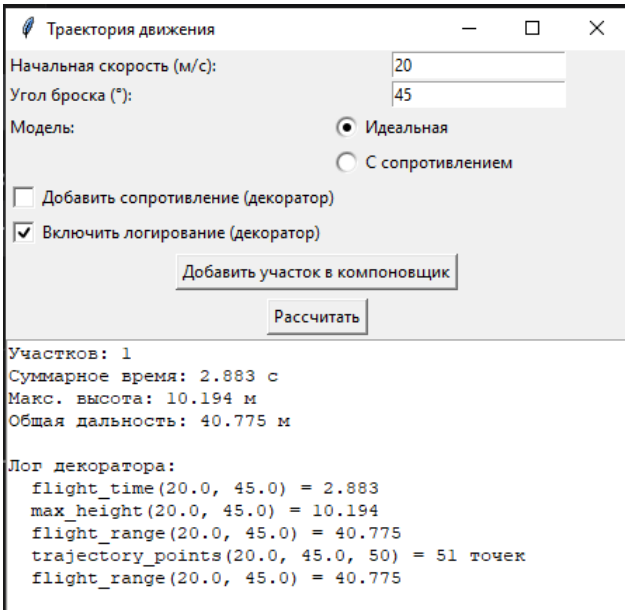


Рисунок 6 — Лог расчётов (LoggingDecorator)

В текстовом поле отображается лог вызовов методов модели, сформированный декоратором.

6. Исходный код

```
import math
import tkinter as tk
from tkinter import messagebox
import matplotlib.pyplot as plt


class TrajectoryComponent:
    """Общий интерфейс для листа и контейнера."""
    def points(self):
        raise NotImplementedError

    def total_time(self):
        raise NotImplementedError
```

```
    def max_height(self):
        raise NotImplementedError

    def flight_range(self):
        raise NotImplementedError


class SimpleSegment(TrajectoryComponent):
    """Лист — один участок траектории."""
    def __init__(self, v0, angle, model):
        self.v0 = v0
        self.angle = angle
        self.model = model

    def points(self):
        return self.model.trajectory_points(self.v0, self.angle, 50)

    def total_time(self):
        return self.model.flight_time(self.v0, self.angle)

    def max_height(self):
        return self.model.max_height(self.v0, self.angle)

    def flight_range(self):
        return self.model.flight_range(self.v0, self.angle)


class CompositeTrajectory(TrajectoryComponent):
    """Контейнер — объединение нескольких участков."""
    def __init__(self):
        self.children = []

    def add(self, component):
        self.children.append(component)

    def remove(self, component):
        self.children.remove(component)

    def points(self):
        result = []
        offset = 0.0
        for child in self.children:
            for (x, y) in child.points():
                result.append((round(x + offset, 3), y))
```

```
            offset += child.flight_range()
        return result

    def total_time(self):
        return sum(child.total_time() for child in self.children)

    def max_height(self):
        return max((child.max_height() for child in self.children),
default=0.0)

    def flight_range(self):
        return sum(child.flight_range() for child in self.children)


class MotionModel:
    def flight_time(self, v0, angle):
        raise NotImplementedError

    def max_height(self, v0, angle):
        raise NotImplementedError

    def flight_range(self, v0, angle):
        raise NotImplementedError

    def trajectory_points(self, v0, angle, steps):
        raise NotImplementedError


class IdealModel(MotionModel):
    def __init__(self, g=9.81):
        self.g = g

    def flight_time(self, v0, angle):
        a = math.radians(angle)
        return 2 * v0 * math.sin(a) / self.g

    def max_height(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g)

    def flight_range(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * math.sin(2 * a) / self.g

    def trajectory_points(self, v0, angle, steps):
```

```
        T = self.flight_time(v0, angle)
        a = math.radians(angle)
        pts = []
        for i in range(steps + 1):
            t = T * i / steps
            x = v0 * math.cos(a) * t
            y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
            pts.append((round(x, 3), round(y, 3)))
        return pts


class BasicDragModel(MotionModel):
    def __init__(self, g=9.81, k=0.05):
        self.g = g
        self.k = k

    def flight_time(self, v0, angle):
        a = math.radians(angle)
        return 2 * v0 * math.sin(a) / (self.g * (1 + self.k))

    def max_height(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g * (1 + self.k))

    def flight_range(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * math.sin(2 * a) / (self.g * (1 + self.k))

    def trajectory_points(self, v0, angle, steps):
        T = self.flight_time(v0, angle)
        a = math.radians(angle)
        pts = []
        for i in range(steps + 1):
            t = T * i / steps
            x = v0 * math.cos(a) * t / (1 + self.k * t)
            y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
            pts.append((round(x, 3), round(max(y, 0), 3)))
        return pts


class MotionDecorator(MotionModel):
    """Абстрактный декоратор."""
    def __init__(self, wrapped: MotionModel):
        self.wrapped = wrapped
```

```
    def flight_time(self, v0, angle):
        return self.wrapped.flight_time(v0, angle)

    def max_height(self, v0, angle):
        return self.wrapped.max_height(v0, angle)

    def flight_range(self, v0, angle):
        return self.wrapped.flight_range(v0, angle)

    def trajectory_points(self, v0, angle, steps):
        return self.wrapped.trajectory_points(v0, angle, steps)


class DragDecorator(MotionDecorator):
    """Декоратор: добавляет учёт сопротивления воздуха."""
    def __init__(self, wrapped: MotionModel, k=0.1):
        super().__init__(wrapped)
        self.k = k

    def flight_time(self, v0, angle):
        return self.wrapped.flight_time(v0, angle) / (1 + self.k)

    def max_height(self, v0, angle):
        return self.wrapped.max_height(v0, angle) / (1 + self.k)

    def flight_range(self, v0, angle):
        return self.wrapped.flight_range(v0, angle) / (1 + self.k)


class LoggingDecorator(MotionDecorator):
    """Декоратор: добавляет логирование расчётов."""
    def __init__(self, wrapped: MotionModel):
        super().__init__(wrapped)
        self.log = []

    def flight_time(self, v0, angle):
        result = self.wrapped.flight_time(v0, angle)
        self.log.append(f"flight_time({v0}, {angle}) = {result:.3f}")
        return result

    def max_height(self, v0, angle):
        result = self.wrapped.max_height(v0, angle)
        self.log.append(f"max_height({v0}, {angle}) = {result:.3f}")
        return result
```

```
        def flight_range(self, v0, angle):
            result = self.wrapped.flight_range(v0, angle)
            self.log.append(f"flight_range({v0}, {angle}) = {result:.3f}")
            return result

        def trajectory_points(self, v0, angle, steps):
            pts = self.wrapped.trajectory_points(v0, angle, steps)
            self.log.append(f"trajectory_points({v0}, {angle}, {steps}) =
{len(pts)} точек")
            return pts

        def get_log(self):
            return self.log


    class TrajectoryApp:
        def __init__(self, root):
            self.root = root
            root.title("Траектория движения")

            self.model = IdealModel()
            self.logging = None
            self.segments = []

            tk.Label(root, text="Начальная скорость (м/с):").grid(row=0,
column=0, sticky="w")
            self.v0_entry = tk.Entry(root)
            self.v0_entry.insert(0, "20")
            self.v0_entry.grid(row=0, column=1)

            tk.Label(root, text="Угол броска (°):").grid(row=1, column=0,
sticky="w")
            self.angle_entry = tk.Entry(root)
            self.angle_entry.insert(0, "45")
            self.angle_entry.grid(row=1, column=1)

            tk.Label(root, text="Модель:").grid(row=2, column=0,
sticky="w")
            self.model_var = tk.StringVar(value="ideal")
            tk.Radiobutton(root, text="Идеальная",
variable=self.model_var,
                           value="ideal",
command=self.update_model).grid(row=2, column=1, sticky="w")
            tk.Radiobutton(root, text="С сопротивлением",
variable=self.model_var,
```

```
                value="drag",
command=self.update_model).grid(row=3, column=1, sticky="w")

        self.drag_var = tk.BooleanVar(value=False)
        tk.Checkbutton(root, text="Добавить сопротивление
(декоратор)",
                variable=self.drag_var).grid(row=4, column=0,
columnspan=2, sticky="w")

        self.log_var = tk.BooleanVar(value=False)
        tk.Checkbutton(root, text="Включить логирование
(декоратор)",
                variable=self.log_var).grid(row=5, column=0,
columnspan=2, sticky="w")

        tk.Button(root, text="Добавить участок в компоновщик",
            command=self.add_segment).grid(row=6, column=0,
columnspan=2, pady=3)
        tk.Button(root, text="Рассчитать",
            command=self.calculate).grid(row=7, column=0,
columnspan=2, pady=3)

        self.result = tk.Text(root, width=55, height=14)
        self.result.grid(row=8, column=0, columnspan=2)

    def update_model(self):
        if self.model_var.get() == "ideal":
            self.model = IdealModel()
        else:
            self.model = BasicDragModel()

    def _build_model(self):
        """Навешиваем декораторы на базовую модель."""
        model = self.model
        if self.drag_var.get():
            model = DragDecorator(model)
        if self.log_var.get():
            self.logging = LoggingDecorator(model)
            model = self.logging
        else:
            self.logging = None
        return model

    def _read_inputs(self):
        v0 = float(self.v0_entry.get())
```

```
        angle = float(self.angle_entry.get())
        if v0 <= 0:
            raise ValueError("Скорость должна быть > 0")
        if not (0 < angle < 90):
            raise ValueError("Угол должен быть в диапазоне (0; 90)")
        return v0, angle

    def add_segment(self):
        """Добавляем очередной участок в компоновщик."""
        try:
            v0, angle = self._read_inputs()
            model = self._build_model()
            segment = SimpleSegment(v0, angle, model)
            self.segments.append(segment)
            messagebox.showinfo("Компоновщик",
                                f"Участок добавлен. Всего участков:
{len(self.segments)}")
        except ValueError as e:
            messagebox.showerror("Ошибка ввода", str(e))

    def calculate(self):
        try:
            if not self.segments:
                v0, angle = self._read_inputs()
                model = self._build_model()
                segment = SimpleSegment(v0, angle, model)
                self.segments.append(segment)

            composite = CompositeTrajectory()
            for seg in self.segments:
                composite.add(seg)

            total_time = composite.total_time()
            height = composite.max_height()
            distance = composite.flight_range()
            pts = composite.points()

            self.result.delete("1.0", tk.END)
            self.result.insert(tk.END,
                f"Участков: {len(self.segments)}\n"
                f"Суммарное время: {total_time:.3f} с\n"
                f"Макс. высота: {height:.3f} м\n"
                f"Общая дальность: {distance:.3f} м\n")

            if self.logging:
```

```
                self.result.insert(tk.END, "\nЛог декоратора:\n")
                for line in self.logging.get_log():
                    self.result.insert(tk.END, "  " + line + "\n")

            xs = [p[0] for p in pts]
            ys = [p[1] for p in pts]
            plt.plot(xs, ys, marker='o', markersize=3)
            plt.title("Составная траектория")
            plt.xlabel("X, м")
            plt.ylabel("Y, м")
            plt.grid(True)
            plt.axis('equal')
            plt.show()
        except ValueError as e:
            messagebox.showerror("Ошибка ввода", str(e))
        except Exception as e:
            messagebox.showerror("Ошибка", str(e))


if __name__ == "__main__":
    root = tk.Tk()
    app = TrajectoryApp(root)
root.mainloop()
```

Вывод:

В ходе выполнения лабораторной работы было разработано приложение «Траектория движения», реализующее расчёт параметров движения тела, брошенного под углом к горизонту.

В приложении применены два структурных паттерна проектирования:

- Компоновщик — для объединения нескольких участков траектории в единую составную траекторию. Клиентский код работает с SimpleSegment и CompositeTrajectory через общий интерфейс TrajectoryComponent, что позволяет строить траектории произвольной сложности.
- Декоратор — для динамического добавления к модели движения дополнительных эффектов. Декораторы DragDecorator и LoggingDecorator оборачивают любую модель (IdealModel, BasicDragModel) и могут комбинироваться, не изменяя базовые классы.

Таким образом, была изучена практическая реализация двух структурных паттернов и получено работоспособное приложение с графическим интерфейсом для расчёта и визуализации траектории движения тела.